

Lab 4. 토픽 → 적응형 카드

처음으로 무언가를 정해 두는 랩입니다

60분 · 트리거 → 입력 변수 → 조건 → 적응형 카드

여기까지는 전부 부탁이었습니다

	무엇을 했나	지켜졌나
Lab 1	지침으로 규칙을 적었다	대체로
Lab 2	지식으로 근거를 쫓다	대체로
Lab 3	웹사이트를 붙였다	근거가 갈렸다

「대체로」가 곤란한 자리가 있습니다. 신청 절차, 담당자 안내, 법적 고지 — 사람마다 답이 달라지면 안 되는 것들입니다.

가장 자주 오해받는 지점부터

토픽

불릴지 말지 — 매번 판단

불린 뒤 — 그린 대로

생성형

불릴지 말지 — 매번 판단

불린 뒤 — 매번 다름

결정론은 토픽 안에서만 성립합니다. 윗줄이 양쪽 다 주황인 것이 오늘 가장 핫갈리는 자리입니다.

이번 랩이 답할 질문

무엇을 어디까지 정해 둘 수 있는가?

- 토픽은 무엇을 보고 불리는가
- 사용자가 말한 값을 어떻게 붙잡아 두는가
- 이미 말한 것을 다시 묻지 않으려면 어떻게 하는가
- 확정된 값과 생성된 문장을 사용자가 구분할 수 있는가

트리거 문구를 나열하는 칸이 없습니다

토픽을 부르는 것은 **작업 설명** 한 문단입니다.

- 「이 에이전트 선택」으로 되어 있고, 아래에 설명 칸만 있습니다
- 문구가 아니라 **뜻으로** 걸립니다 — 설명에 없는 표현이어도 이어집니다
- **Lab 2** 16번에서 문서마다 설명을 적었던 것과 같은 구조입니다

오케스트레이터는 지식이든 도구든 토픽이든 **설명을 읽고 무엇을 쓸지 고릅니다**. 오늘 세 번째로 나오는 얘기입니다.

설명 한 칸을 바꾸면 불리지 않습니다

노드도, 메시지도, 토픽 이름도 그대로입니다.

잘 만든 토픽이 동작하지 않을 때 노드부터 뒤지기 쉽습니다. **먼저 볼 곳은 설명입니다.**

- 짧게 뭉개는 것으로는 잘 안 됩니다 — 토픽 이름과 앞뒤 맥락으로 걸어냅니다
- **다른 곳을 가리켜야** 결과가 갑니다

같은 문장인데 답이 다릅니다

첫 질문일 때

참고할 이력이 없다

가장 엉뚱하게 가기 쉽습니다

이어서 물을 때

「그 답은 이미 했다」가 붙는다

다른 쪽으로 넘깁니다

오케스트레이터는 매 턴 **대화 이력까지 포함해** 다시 판단합니다. 두 번째 질문은 첫 번째와 같은 입력이 아닙니다.

지침 한 줄로 고정합니다 — 대가는 있습니다

연차·휴가 관련 질문은 /연차 신청 방법 토픽으로 수행한다.

- 4번의 설명이 「이 토픽은 무엇을 하는가」라면, 이 줄은 「이럴 때는 저 토픽을 써라」고 시키는 것입니다
- 판정 근거를 늘리는 것과 실행을 지시하는 것은 다릅니다
- / 는 Lab 5에서 도구를 지목할 때도 같은 자리에 씁니다

⚠ 이 지침은 「연차, 휴가 관련 질문」 전부를 토픽으로 보냅니다. 지식이 답해야 할 질문까지 가져갑니다.

사용자의 대답이 이름 붙은 값으로 남습니다

여기서 만든 변수가 이 랩의 진짜 산출물입니다.

- 대답이 문장 속으로 사라지지 않습니다
- `var1` · `var2` 로 두면 토픽이 커졌을 때 어느 값인지 알 수 없습니다 — 만든 자리에서 바로 바꿉니다
- Lab 5에서는 이 값으로 실제 Excel을 조회하고, Lab 6에서는 흐름에 넘깁니다

⚠ 타이핑한 `{문의유형}` 은 변수가 아니라 글자입니다. `{x}` 로 넣어야 칩으로 표시되고, 그 표시가 없으면 연결되지 않은 것입니다.

비어 있을 때만 묻습니다

담당자 알려줘
값이 비어 있다
질문 노드가 묻습니다

복지포인트 담당자 알려줘
오케스트레이터가 미리 채웠다
그냥 지나갑니다

노드를 지운 것이 아닙니다. 화면의 [참조](#) 항목에 「변수가 비어 있을 경우 이 노드에서 값을 가져옵니다」라고 적혀 있습니다.

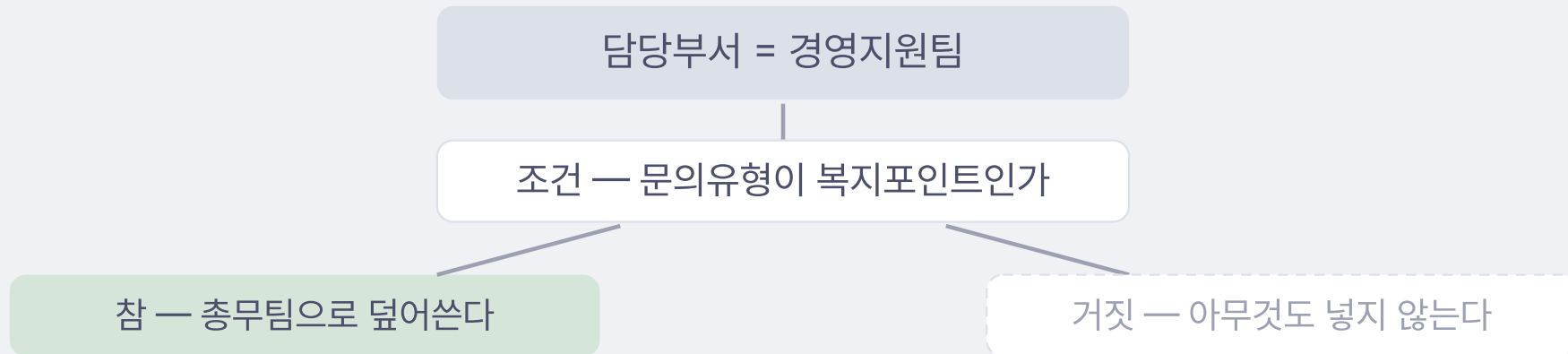
말기면 흔들린다고 했는데

세 번의 대화에서 오케스트레이터가 값을 채우고 · 유지하고 · 비웁니다.

이 랩에서 오케스트레이터가 흔들리지 않은 첫 대목입니다. 11번에서는 같은 질문에 답이 같았고 14번에서는 첫 질문이 엉뚱하게 갔는데, 여기서는 말긴 일을 그대로 해냅니다.

- 값을 말하면 채웁니다
- 다음 턴까지 유지합니다
- 바뀌었다는 신호가 오면 비웁니다

기본값을 먼저 두고 예외만 덮어씹습니다



갈래가 늘어나도 손볼 곳은 **예외 쪽**입니다. ⚠ 변수는 갈래에 들어가기 **전에** 만듭니다.

카드는 보여주는 노드가 아니라 묻는 노드입니다

	어떤 노드인가
메시지 보내기	글을 내보내는 노드
적응형 카드로 질문	묻는 노드 — 17번의 「질문하기」와 같은 갈래

메시지 노드를 카드로 바꿀 수는 없습니다. 새 노드로 넣어야 합니다.

- 이름이 「질문」인데 **묻지 않고 보여주기만 할 수도** 있습니다. 오늘은 그렇게 씁니다
- **⚠ 이 노드에는 슬롯 필링이 걸리지 않습니다.** 25~27번에서 한 것은 「질문하기」에서만 됩니다

카드에 변수를 넣으려면 수식이어야 합니다

`${문의유형}` 을 그대로 두면 **글자로 찍힙니다.**

- JSON은 값을 계산하지 않고 **적힌 대로** 씁니다
-  여기에는 `{x}` 버튼이 없습니다 — 22번의 메시지 노드와 달리 이름을 직접 씁니다
- `문의유형` 만 `Text()` 로 감쌉니다. `choice` 형식이라 문자열로 바뀌어야 합니다

22번에서 두 변수의 형식이 다르게 표시된 것이 여기서 걸립니다 — `문의유형` 은 `choice` , `담당부서` 는 `string` .

카드가 중요한 이유는 모양이 아닙니다

어디까지가 에이전트가 **확정한 값인지**를 화면에서 갈라 줍니다.

- 문장 속 숫자는 생성된 것처럼 보입니다
- 카드의 필드 값은 **채워 넣은 것**으로 읽힙니다
- 같은 값이라도 사용자가 받아들이는 무게가 다릅니다

오늘 만든 것이 이 한 화면에 다 있습니다 — 27번의 슬롯 필링이 질문을 건너뛰었고, 30번의 조건이 담당부서를 정했고, 35번의 수식이 그 값을 카드에 넣었습니다.

맡기면 흔들리고, 정해 두면 고정됩니다

오늘 본 것	어디서
같은 질문에 답이 같았다	11번
첫 질문이 엉뚱하게 갔다	14번
지침 한 줄로 고정됐다	13번
맡긴 값을 정확히 관리했다	32번

흔들려도 되는 것은 맡기고, 흔들리면 안 되는 것은 정해 둡니다. Lab 5에서는 바깥에 손을 뻗습니다 — 거기서 이 판단이 훨씬 무거워집니다.